



DAAN862: ANALYTICS PROGRAMMING IN PYTHON

Lesson 3: Statistical Analysis with Pandas

Lesson 3.1: Basic Data Types: Series

In this course, the import conventions shown in Figure 3.1 are used for pandas.

```
In [1]: from pandas import Series, DataFrame  
In [2]: import pandas as pd
```

Fig 3.1 (click to enlarge)

pd is a common short name for pandas. You can also import *Series* and *DataFrame* into local name space such that you can directly use function *Series* or *DataFrame* instead of *pd.Series* or *pd.DataFrame* which are only used for convenience, however both ways are acceptable.

A **Series** is a one-dimensional array-like object containing an array of data (of any NumPy data type) and has an associated array of data labels, called its **index**. The simplest Series is formed from only an array of data:

```
In [3]: s1 = Series(range(5))  
In [4]: s1  
Out[4]:  
0    0  
1    1  
2    2  
3    3  
4    4  
dtype: int64
```

Index	Value
0	0
1	1
2	2
3	3
4	4

Fig 3.2

The string representation of a series displayed in Fig 3.2 interactively shows the index on the left and the values on the right. Since we did not specify an

index of data, a default one consisting of the integers 0 through N-1 (where N is the length of the data) is created.

You can customize this index by assigning a list of strings to the index parameter in Series function:

```
In [6]: S2 = Series(range(5), index = ['a', 'b', 'c', 'd', 'e'])  
In [7]: S2  
Out[7]:  
a      0  
b      1  
c      2  
d      3  
e      4  
dtype: int64
```

Fig 3.3 (click to enlarge)

As highlighted in Figure 3.3, the indexes are ['a','b','c','d','e'].

As showing in Figure 3.4, series can be indexed similarly to list or array. It can also be indexed by the values in the index:

```
In [8]: S2['c']          # Select row 'c'  
Out[8]: 2  
  
In [9]: S2['e'] = 100    # Assign 100 to row 'e'  
  
In [10]: S2[['a', 'c', 'e']] # Select row 'a', 'c' and 'e'  
Out[10]:  
a      0  
c      2  
e     100  
dtype: int64
```

Fig 3.4 (click to enlarge)

As shown in Figure 3.5, similar logic comparison and math operations can be applied to Series:

```
In [11]: S2[S2 > 2]           # Select all elements larger than 2
Out[11]:
d      3
e    100
dtype: int64
```

Fig 3.5 (click to enlarge)

As showing in Figure 3.6, you can multiply 10 to all elements:

```
In [12]: S2 * 10             # Multiply 10 to all elements
Out[12]:
a      0
b     10
c     20
d     30
e   1000
dtype: int64
```

Fig 3.6 (click to enlarge)

As shown in Figure 3.7, the NumPy functions can be applied to Series too. For example, you can calculate the square root of all elements:

```
In [13]: np.sqrt(S2)        # Calculate the square root of all elements.
Out[13]:
a    0.000000
b    1.000000
c    1.414214
d    1.732051
e   10.000000
dtype: float64
```

Fig 3.7 (click to enlarge)

Table 3.1.1: The Important Attributes of Series

Attributes	Description
T	Return the transpose, which is by definition self
at	Fast label-based scalar accessor

axes	Return a list of the row axis labels
dtype	Return the dtype object of the underlying data
empty	Return Boolean if the series is empty
hasnans	Return Boolean if the series contain nans
iat	Fast integer location scalar accessor
iloc	Purely integer-location based indexing for selection by position
is unique	Return boolean if values in the object are unique
loc	Purely label-location based indexer for selection by label
ndim	Return the number of dimensions of the underlying data
shape	Return a tuple of the shape of the underlying data
size	Return the number of elements in the underlying data
values	Return Series as ndarray or ndarray-like



Example

Click the dropdown to view examples on how to use some of the attributes shown in Table 3.1.1

```

In [14]: S2.axes                # Return row index labels
Out[14]: [Index(['a', 'b', 'c', 'd', 'e'], dtype='object')]

In [15]: S2.empty              # Check if the Series is empty
Out[15]: False

```

```
In [16]: S2.hasnans          # Check if the Series has NaNs
Out[16]: False

In [17]: S2.shape           # Shape of the Series
Out[17]: (5,)

In [18]: S2.size            # The number of elements
Out[18]: 5
```

Fig 3.8 (click to enlarge)